

LECTURE 19

The Normalization Process: 1NF, 2NF, 3NF

One running example, carried through every stage by hand

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

A Repeatable Procedure, Applied Start to Finish

- 1 First Normal Form (1NF) — atomic values, no repeating groups
- 2 Second Normal Form (2NF) — no partial dependency on a composite key
- 3 Third Normal Form (3NF) — no transitive dependency
- 4 One running example, converted step by step: UNF $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF
- 5 A second, shorter worked example to reinforce the pattern
- 6 Testing for normal forms — the checklist to apply to any relation

An Unnormalized Order Table (UNF)

One row per order, with product lines packed into a repeating group inside the row:

orderNo	custNo	custName	{ productNo, productName, unitPrice, qty }
O100	C001	Ann Beech	{ (P01,Widget,9.99,3), (P02,Gadget,24.99,1) }
O101	C002	Tom Kelly	{ (P01,Widget,9.99,2) }
O102	C001	Ann Beech	{ (P03,Sprocket,4.50,10) }

WARNING

Order O100's single row holds TWO product lines inside one cell — exactly the “repeating group” Lecture 5 ruled out. This is not yet a valid relation at all — it is Unnormalized Form (UNF), the raw starting point.

orderNo → orderDate, custNo · custNo → custName, custAddress
productNo → productName, unitPrice · { orderNo, productNo } → qty

First Normal Form (1NF)

NOTE

A relation is in 1NF if every attribute holds a single, atomic value — no repeating groups, no nested tables. This is simply Lecture 5's definition of a valid relation, restated as the first checkpoint.

Flatten: give each product line its own row, repeating order-level attributes. A single orderNo no longer identifies one row — the primary key grows composite: {orderNo, productNo}.

Order (orderNo, productNo, orderDate, custNo, custName, custAddress, productName, unitPrice, qty) — 1NF

orderNo	productNo	custNo	custName	productName	unitPrice	qty
O100	P01	C001	Ann Beech	Widget	9.99	3
O100	P02	C001	Ann Beech	Gadget	24.99	1
O101	P01	C002	Tom Kelly	Widget	9.99	2
O102	P03	C001	Ann Beech	Sprocket	4.50	10

1NF fixed the STRUCTURAL problem. It did nothing about the redundancy problem — orderDate/custName/custAddress repeat across O100's two rows; productName/unitPrice for Widget repeat across O100 and O101.

1NF: Why the Key Had to Grow

CHECK

In the UNF table, orderNo alone identified a row. After flattening, O100 appears on TWO rows, so orderNo is no longer unique.

Smallest key that identifies every row: {orderNo, productNo}. (O100, P01) and (O100, P02) are different rows; neither attribute alone works.

Question: is orderNo alone a key of the 1NF table?

orderNo	productNo	custNo	custName	productName	unitPrice	qty
O100	P01	C001	Ann Beech	Widget	9.99	3
O100	P02	C001	Ann Beech	Gadget	24.99	1
O101	P01	C002	Tom Kelly	Widget	9.99	2
O102	P03	C001	Ann Beech	Sprocket	4.50	10

Answer: No. O100 repeats. So the key is composite, and orderDate, custNo, custName, and custAddress now repeat inside it. That is the 2NF problem.

Second Normal Form (2NF): Testing for Partial Dependencies

NOTE

2NF: already in 1NF, and every non-key attribute is fully functionally dependent on the WHOLE primary key — not only part of it. A partial dependency: a non-key attribute depends on a proper subset of a composite key.

Key = {orderNo, productNo}. Check each non-key attribute against both halves:

Non-key attribute	Depends on orderNo alone?	Depends on productNo alone?	Full key?
orderDate	Yes	No	Partial
custNo	Yes	No	Partial
custName / custAddress	Yes (via custNo)	No	Partial
productName	No	Yes	Partial
unitPrice	No	Yes	Partial
qty	No	No	Full — needs BOTH

Six of seven non-key attributes are partially dependent — only qty needs the whole key. This relation VIOLATES 2NF.

2NF: Proving One Partial Dependency

PROOF

Test orderDate against each half of the key. Does orderNo alone fix it? Look at the rows:

orderNo O100 gives 2026-08-01 on BOTH rows: the same value every time. So orderDate depends on orderNo alone, a partial dependency.

orderNo	productNo	orderDate
O100	P01	2026-08-01
O100	P02	2026-08-01
O101	P01	2026-08-02
O102	P03	2026-08-03

productNo P01 gives 2026-08-01 (O100) and 2026-08-02 (O101): different dates, so productNo does NOT determine orderDate.

Converting to 2NF

Every partially-dependent attribute moves to a relation keyed by whichever part of the composite key determines it:

Order (orderNo, orderDate, custNo, custName, custAddress)

Product (productNo, productName, unitPrice)

OrderLine (orderNo, productNo, qty)

orderNo	orderDate	custNo	productNo	productName	unitPrice	orderNo	productNo	qty
O100	2026-08-01	C001	P01	Widget	9.99	O100	P01	3
O101	2026-08-02	C002	P02	Gadget	24.99	O100	P02	1
O102	2026-08-03	C001	P03	Sprocket	4.50	O101	P01	2
						O102	P03	10

TIP

Product and Order now have single-attribute keys, so partial dependency is impossible there. qty needs the whole key.

Order still has a problem: does custName depend on orderNo, or on custNo?

Third Normal Form (3NF): Transitive Dependency

NOTE

3NF: already in 2NF, and no non-key attribute is transitively dependent on the primary key. Transitive dependency: Z depends on non-key Y, and Y depends on key X — $X \rightarrow Y \rightarrow Z$, but Z doesn't depend on X directly.

Check Order(orderNo, orderDate, custNo, custName, custAddress), key orderNo: $\text{custNo} \rightarrow \text{custName}, \text{custAddress}$ — but custNo is itself a NON-KEY attribute here. So custName/custAddress depend on orderNo only transitively: $\text{orderNo} \rightarrow \text{custNo} \rightarrow \text{custName}$. VIOLATES 3NF.

Customer (custNo, custName, custAddress)

custNo	custName	custAddress
C001	Ann Beech	12 Elm St, Lahore
C002	Tom Kelly	5 Oak Ave, Karachi

Order (orderNo, orderDate, custNo)

orderNo	orderDate	custNo
O100	2026-08-01	C001
O101	2026-08-02	C002
O102	2026-08-03	C001

Split off the transitively-dependent attributes into Customer, keyed by custNo, leaving custNo behind in Order as a foreign key. Product and OrderLine carry over unchanged.

3NF: Proving the Transitive Chain

PROOF

Start from Order(orderNo, orderDate, custNo, custName, custAddress), key orderNo. Follow the chain:

Step 1: orderNo → custNo (one customer per order). Step 2: custNo → custName, custAddress (one name per customer). Step 3: so custName depends on orderNo only THROUGH custNo: transitive. VIOLATES 3NF.

Order: Ann Beech is stored twice (O100, O102)

orderNo	custNo	custName
O100	C001	Ann Beech
O101	C002	Tom Kelly
O102	C001	Ann Beech

After the split: each customer is stored once

custNo	custName	custAddress
C001	Ann Beech	12 Elm St, Lahore
C002	Tom Kelly	5 Oak Ave, Karachi

Splitting off Customer removes the repeat; Order keeps custNo as a foreign key.

Final 3NF Schema — Verified BCNF Too

Customer (custNo, custName, custAddress)

Product (productNo, productName, unitPrice)

Order (orderNo, orderDate, custNo)

Foreign Key custNo references Customer(custNo)

OrderLine (orderNo, productNo, qty)

FK orderNo → Order, FK productNo → Product

UNF

One row per order,
product lines repeat
inside it

1NF

Flattened — one row
per (order, product); key
grows composite

2NF

Split off attributes
depending on only half
the composite key

3NF

Split off attributes
depending on a non-key
attribute, not the key

What Each Stage Removed

Each stage fixed one specific kind of problem. Nothing was lost along the way:

Stage	Problem found	Attributes involved	Fix
UNF → 1NF	Repeating group in one cell	productNo, productName, unitPrice, qty	Flatten to one row per (order, product)
1NF → 2NF	Partial dependency on half the key	orderDate, custNo, custName, custAddress on orderNo; productName, unitPrice on productNo	Move each group to Order or Product
2NF → 3NF	Transitive dependency through custNo	custName, custAddress via orderNo → custNo → custName	Move to Customer; keep custNo in Order

Every Anomaly Gone — At No Cost

- Product accepts new, unordered product P04 — insertion anomaly gone
- Deleting O101 leaves Customer row C002 intact — deletion anomaly gone
- Widget's price lives in exactly one row of Product — modification anomaly gone

Customer ⋈ Order ⋈ OrderLine ⋈ Product reconstructs the original flattened 1NF table exactly

Every join attribute (custNo, orderNo, productNo) is the primary key of the relation on the “one” side of the join — which is exactly why this decomposition is lossless.

Hand-Tracing the Lossless Join

Rejoin the four 3NF relations one step at a time. Each join matches on a primary key of the "one" side:

Step	Join performed	Rows produced	Check
1	Order ⋈ Customer on custNo	3 rows: O100, O101, O102, each with its name and address	Matches 3 orders
2	... ⋈ OrderLine on orderNo	4 rows: O100 (2 lines), O101 (1), O102 (1)	Matches 4 order lines
3	... ⋈ Product on productNo	4 rows with productName and unitPrice attached	Equals original 1NF table

Enrollment: Confirming the Pattern Generalizes

Enrollment (studentNo, courseNo, studentName, courseName, instructorNo, instructorOffice, grade) — 1NF

studentNo	courseNo	studentName	courseName	instructorNo	instructorOffice	grade
S1	CS201	Bilal Khan	Databases	I01	Room 214	A
S1	CS305	Bilal Khan	Networks	I02	Room 108	B+
S2	CS201	Sara Malik	Databases	I01	Room 214	A-

FDs: studentNo $\rightarrow$ studentName; courseNo $\rightarrow$ courseName, instructorNo; instructorNo $\rightarrow$ instructorOffice; {studentNo, courseNo} $\rightarrow$ grade

- 2NF check vs {studentNo, courseNo}: studentName depends on studentNo alone; courseName and instructorNo on courseNo alone; instructorOffice follows instructorNo. Only grade needs both. VIOLATES 2NF.

Student(studentNo,studentName) Instructor(instructorNo,instructorOffice)
Course(courseNo,courseName,instructorNo) Enrollment(studentNo,courseNo,grade)

Enrollment: Splitting on 2NF Violations

Enrollment (studentNo, courseNo, studentName, courseName, instructorNo, instructorOffice, grade) — 1NF

studentNo	courseNo	studentName	courseName	instructorNo	instructorOffice	grade
S1	CS201	Bilal Khan	Databases	I01	Room 214	A
S1	CS305	Bilal Khan	Networks	I02	Room 108	B+
S2	CS201	Sara Malik	Databases	I01	Room 214	A-

FDs: studentNo $\rightarrow$ studentName; courseNo $\rightarrow$ courseName, instructorNo; instructorNo $\rightarrow$ instructorOffice; {studentNo, courseNo} $\rightarrow$ grade

- 2NF test against {studentNo, courseNo}: studentName depends on studentNo alone; courseName and instructorNo on courseNo alone; instructorOffice follows instructorNo. Only grade needs both.

Result: Student(studentNo, studentName), Course(courseNo, courseName, instructorNo, instructorOffice),
Enrollment(studentNo, courseNo, grade)

Enrollment: Removing the Transitive Dependency

Course(courseNo, courseName, instructorNo, instructorOffice), key courseNo. Follow the chain:

courseNo	courseName	instructorNo	instructorOffice
CS201	Databases	I01	Room 214
CS305	Networks	I02	Room 108

Transitive: courseNo $\rightarrow$ instructorNo $\rightarrow$ instructorOffice. instructorOffice depends on courseNo only through instructorNo, a non-key attribute of Course.

- Split: Instructor(instructorNo, instructorOffice) and Course(courseNo, courseName, instructorNo). Now every FD has a key on its left.

Final: Student(studentNo, studentName), Instructor(instructorNo, instructorOffice), Course(courseNo, courseName, instructorNo), Enrollment(studentNo, courseNo, grade)

Testing Any Relation for Normal Forms

1. Is every attribute value atomic?

No repeating groups, no multi-valued cells. No $\rightarrow$ not even 1NF; flatten first.



2. Is the key composite? Does every non-key need the **WHOLE** key?

Single-attribute key $\rightarrow$ automatically 2NF. Composite $\rightarrow$ check each attribute against each proper subset.



3. Does every non-key attribute depend **ONLY** on the key?

If X (key) $\rightarrow Y$ (non-key) $\rightarrow Z$ (non-key), Z is transitively dependent $\rightarrow$ violates 3NF.

WARNING

Partial vs. transitive — the most common exam mix-up: partial is about a composite key (some attribute depends on only part of it). Transitive can happen even with a single-attribute key — the culprit is a non-key attribute standing in the way. If the key is single-attribute, only check for transitive dependency.

Common Mistakes When Normalizing

1. Calling every violation "partial"

Partial needs a COMPOSITE key. In a relation keyed by orderNo alone, custNo $\rightarrow$ custName is transitive, not partial.



2. Testing sample rows instead of business rules

Two rows agreeing on custName do not prove custNo $\rightarrow$ custName. The FD must hold for every legal instance.



3. Dropping the foreign key after a split

Order keeps custNo so it can still reach Customer. Dropping it breaks the lossless join.

WARNING

After every split, rejoin and compare to the original. If the join rebuilds the original rows exactly, the split is safe.

KEY TAKEAWAYS

Key Takeaways

- 1 1NF: every attribute value is atomic — fixed by flattening, which often forces the key to become composite
- 2 2NF: every non-key attribute depends on the WHOLE composite key, not part of it — relevant only for composite keys
- 3 3NF: every non-key attribute depends DIRECTLY on the key, never transitively through another non-key attribute
- 4 The running Order example went UNF $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF, ending at Customer, Product, Order, OrderLine — zero redundancy, verified lossless
- 5 Testing any relation: atomicity, then partial dependency (composite keys only), then transitive dependency — in that order
- 6 3NF eliminates most redundancy, but not all — Lecture 20 introduces BCNF, a stricter test